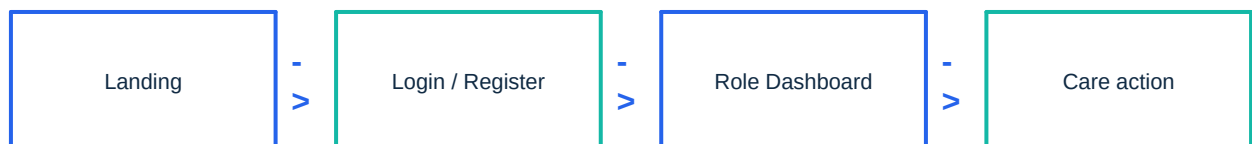
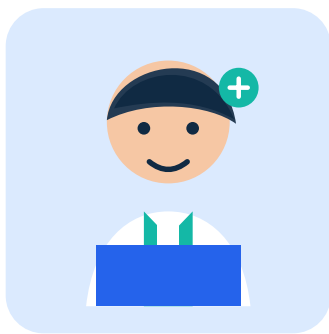


DentiFlow

System Workflow and Architecture Guide

A plain-language reference for developers, designers, clinic staff, QA, and project reviewers.



Purpose. DentiFlow connects reception, doctors, patients, clinical records, treatment plans, billing, inventory, and follow-ups in one Flask + MySQL application.

Important. Seeded clinical information is demonstration data and must not be treated as a confirmed diagnosis or used for real patient care.

1. What the system does

A person enters through the landing page, signs in or creates an account, and is taken to a role-specific workspace. Clinic staff coordinate visits; doctors review patients and record care; patients see only their own care information.

Doctor Reviews assigned patients, appointments, dental findings, periodontal notes, prescriptions, treatment plans, and invoices.	Patient Views their own dashboard, appointments, records, treatment plan, prescriptions, and bills.
Reception / Admin Manages registration, appointments, queue operations, branches, staff, inventory, reports, settings, and billing.	MySQL Stores users, patients, doctors, appointments, queue tokens, clinical records, plans, invoices, payments, and audits.

2. System architecture



The browser sends normal page requests and fetch requests. Flask authenticates the session, validates the request, calls SQLAlchemy, and returns HTML or JSON. PyMySQL connects SQLAlchemy to the existing MySQL database. Local cartoon avatars are served from /static/img. Passwords are stored as hashes.

Request lifecycle

User clicks a link or submits a form.

Flask matches the URL to a blueprint route.

Flask-Login checks the session and role.

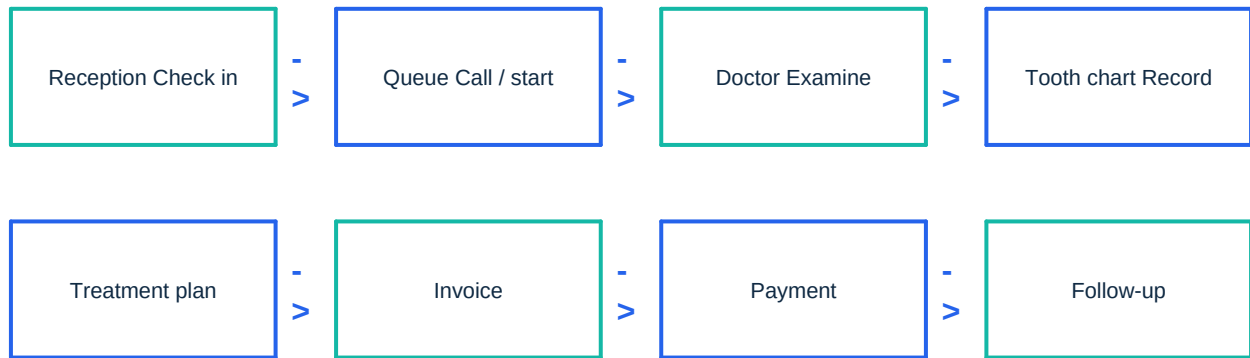
The route reads or changes SQLAlchemy model objects.

SQLAlchemy reads or writes MySQL.

Flask returns HTML or JSON.

JavaScript renders success, empty, loading, or error states.

3. Complete clinic workflow



Doctor flow: Aravind example

Doctor signs in -> Dashboard -> Patients -> Aravind -> Clinical profile -> Dental Chart -> tooth 16 -> clinical finding -> Treatment Plan -> Invoice -> Payment -> Follow-up.

Patient flow

Patient signs in -> Patient Dashboard -> next appointment -> My Records -> Dental History -> Treatment Plan -> Prescription -> Bill. Patient navigation does not expose staff, inventory, other patients, or clinic-wide analytics.

4. Main route map

Area	Routes	Purpose
Landing	/	Public welcome or authenticated dashboard.
Auth	/login, /logout, /register	Sign in, sign out, create doctor/patient account.
Dashboard	/dashboard, /api/dashboard	Role dashboard and operational JSON.
Patients	/patients, /patients/<id>	Directory and clinical profile.
Clinical	/clinical, /api/patients/<id>/teeth	Dental chart, perio, prescriptions, voice notes.
Operations	/appointments, /queue, /staff	Visits, queue, care team.
Finance	/treatment-plans, /billing	Plans, invoices, receipts, payments.
Health	/api/health, /favicon.ico	MySQL health and brand icon.

5. Data model map

Model	What it stores	Connected records
User	Email, password hash, role, branch, avatar, login identity.	Doctor profile, audit logs.
Patient	Demographics, contact, balance, alert state.	Appointments, findings, prescriptions, plans, invoices, X-rays.
Doctor	Specialty, experience, branch, avatar, rating.	Appointments, patients, shifts.
Appointment / QueueToken	Visit date, status, procedure, emergency priority, queue state.	Patient, doctor, chair.
Clinical records	ToothFinding, PeriodontalRecord, ClinicalNote, Prescription, XRayImage.	Patient and doctor.
Finance	TreatmentPlan, phases, items, Invoice, Payment, EMIPlan.	Patient, doctor, appointment.

6. Security and persistence rules

- Passwords are hashed with Werkzeug; plain passwords are not stored.
- MySQL remains the configured database; normal startup does not drop tables.
- Destructive demo reset requires explicit opt-in.
- Patient HTML and API detail routes reject another patient record with HTTP 403.
- API failures return friendly JSON messages instead of raw tracebacks.
- Profile images are local cartoon SVG assets; clinical X-rays remain separate clinical records.

7. Team responsibilities

Team	Owns	Done means
Frontend	Templates, CSS, JavaScript, responsive states.	Loading, success, empty, error, and mobile states work.
Backend	Routes, auth, permissions, JSON contracts.	Status codes, validation, no unauthorized records.
Data	Models, MySQL persistence, seed data.	Changes survive restart; reset is explicit.
QA	Route, browser, API, responsive checks.	Login, dashboard, patient, chart, billing, health pass.
Design	Visual language, cartoons, hierarchy, accessibility.	No broken images; readable desktop/mobile layouts.

8. Run and test

Install: `pip install -r requirements.txt`. Start: `python app.py`. Open: `http://127.0.0.1:5000/`. Health: `http://127.0.0.1:5000/api/health` should return status ok, database mysql, and database_connected true. Demo doctor: `doctor@dentiflow.com / doctor123`.

Generated for the DentiFlow internship demonstration. Clinical records shown in the prototype are demo data only.